

Aula 8

- A interface SPI (*Serial Peripheral Interface*)
 - Sinalização e operação
 - Arquitetura de ligação
 - Tipos de transferências
- A interface I²C (*Inter-Integrated Circuit*)
 - Caraterísticas básicas e sinalização
 - Endereçamento
 - Transferência de dados
 - Múltiplos Masters

José Luís Azevedo, Bernardo Cunha, Tomás O. Silva, P. Bartolomeu

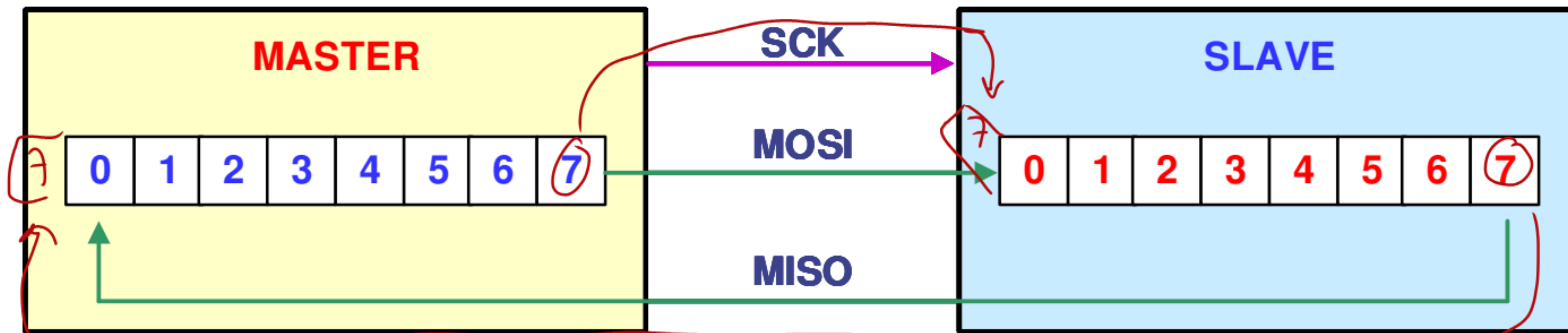
SPI

- SPI – "Serial Peripheral Interface"
- Interface definida inicialmente pela Motorola (evolução do Microwire da National Semiconductor, que é um *subset* do protocolo SPI)
- O SPI é utilizado para comunicar com ou entre uma grande variedade de dispositivos:
 - Sensores de diversos tipos: temperatura, pressão, ...
 - Cartões de memória (MMC / SD) → Não usam SPI nativamente, preciso fazer arranque especial
 - Circuitos: memórias, ADCs, DACs, Displays LCD, OLED
 - Comunicação entre microcontroladores
microcontroladores – sensores
- Ligação a curtas distâncias (dezenas de cm)

SPI

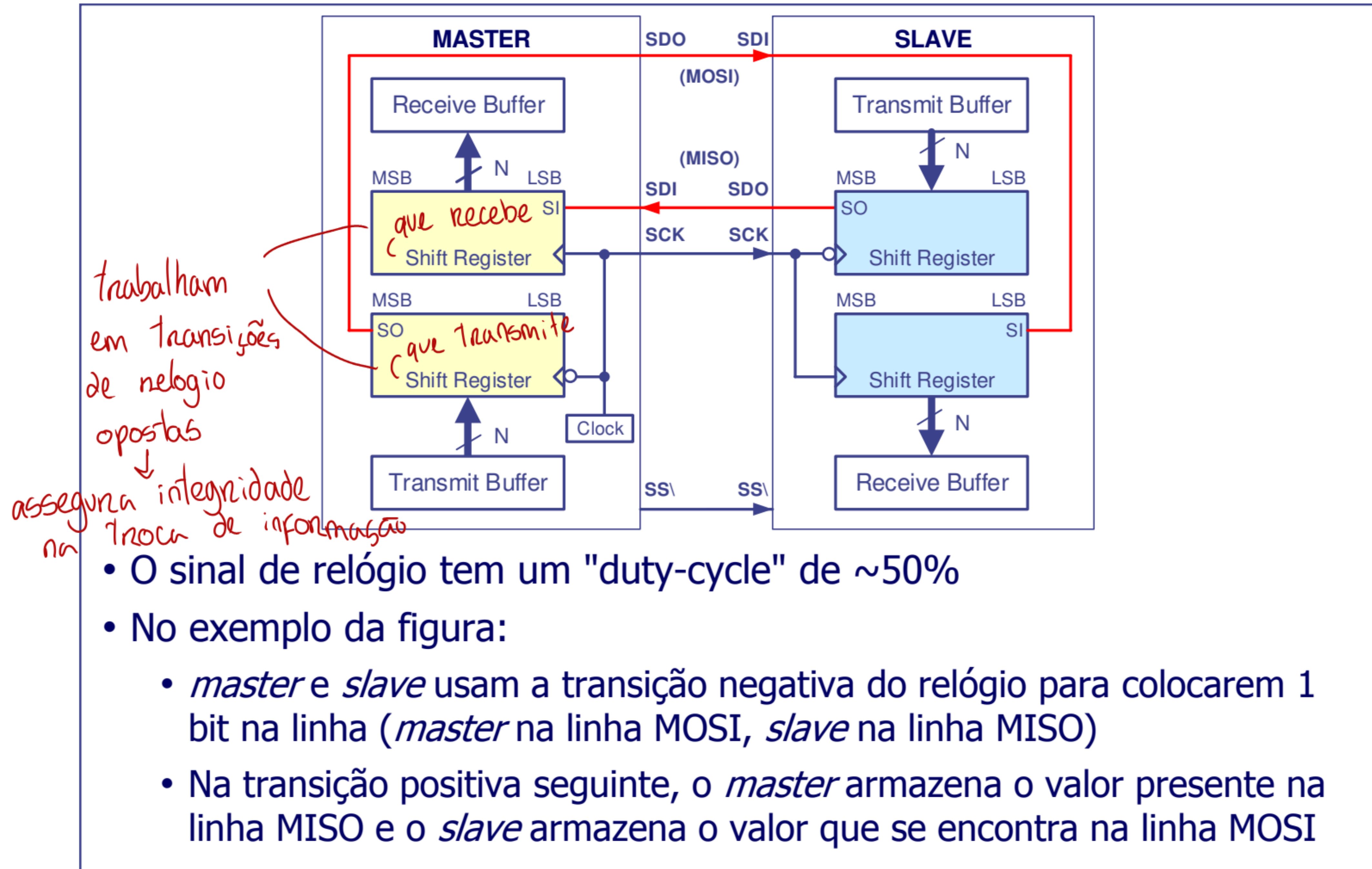
- Arquitetura "Master-Slave" com ligação ponto a ponto
 - O sistema só pode ter um *master*, que é o único dispositivo no sistema que pode controlar o relógio *↳ fixo*
 - Um *master* pode estar ligado a vários *slaves*, para cada comunicação, apenas um *slave* é selecionado pelo *master*, resultando numa ligação ponto a ponto *master ligado a varios slaves mas comunica com um de cada vez*
 - O *master* inicia e controla a transferência de dados
- Comunicação bidirecional "full-duplex" *↳ entre dispositivos nos dois sentidos*
- Transmissão síncrona (relógio explícito): o relógio é sempre gerado pelo *master* que o disponibiliza para todos os *slaves*
- Sinalização:
 - **SCK** – clock
 - Relógio gerado pelo *master* que sincroniza a transmissão/receção de dados
 - **MOSI** – Master Output Slave Input (SDO no *master*) *dados do master para slave*
 - Linha do *master* para envio de dados para o *slave*
 - **MISO** – Master Input Slave Output (SDI no *master*) *dados do slave para o master*
 - Linha do *slave* para enviar dados para o *master*
 - **SS** – Slave select – *para selecionar o slave com quem comunicar*
 - Linha do *master* que seleciona o *slave* com quem vai comunicar *↳ existe uma por dispositivo*

SPI – princípio de funcionamento



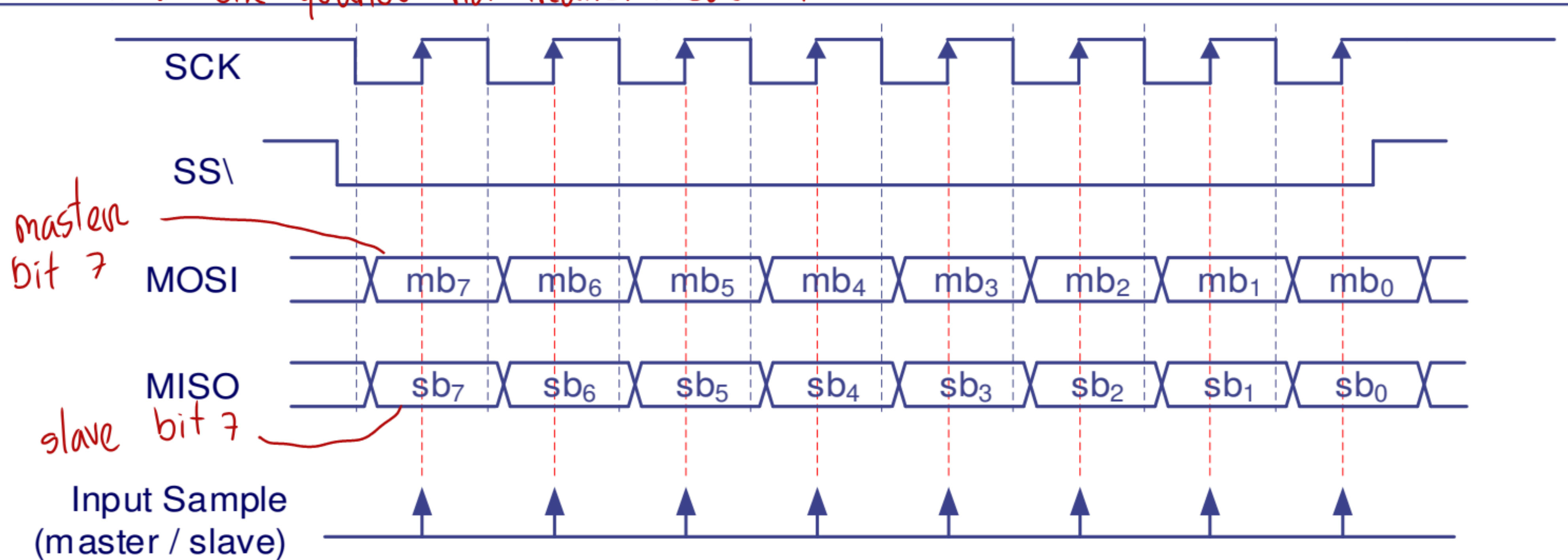
- Transmissão "full-duplex" baseada em dois *shift-registers* (um no *master* e outro no *slave*)
- Em cada ciclo de relógio: *começa pelo mais significativo*
 - O *master* coloca 1 bit na linha MOSI e o *slave* coloca 1 bit na linha MISO
 - Simultaneamente, o *slave* lê o bit presente em MOSI e o *master* lê o bit presente em MISO
- Após N ciclos de relógio o *master* enviou uma palavra de N bits e recebeu do *slave* uma palavra com a mesma dimensão – processo conhecido como "Data Exchange" *↪ Trocaram 1 palavra (de 8 bits)*
- Esta sequência é realizada mesmo quando é pretendida uma comunicação unidirecional (*master* -> *slave* ou *slave* -> *master*)

SPI – princípio de funcionamento



Operação – exemplo

só há clk quando há transmissão (quando estável n há tanto consumo)



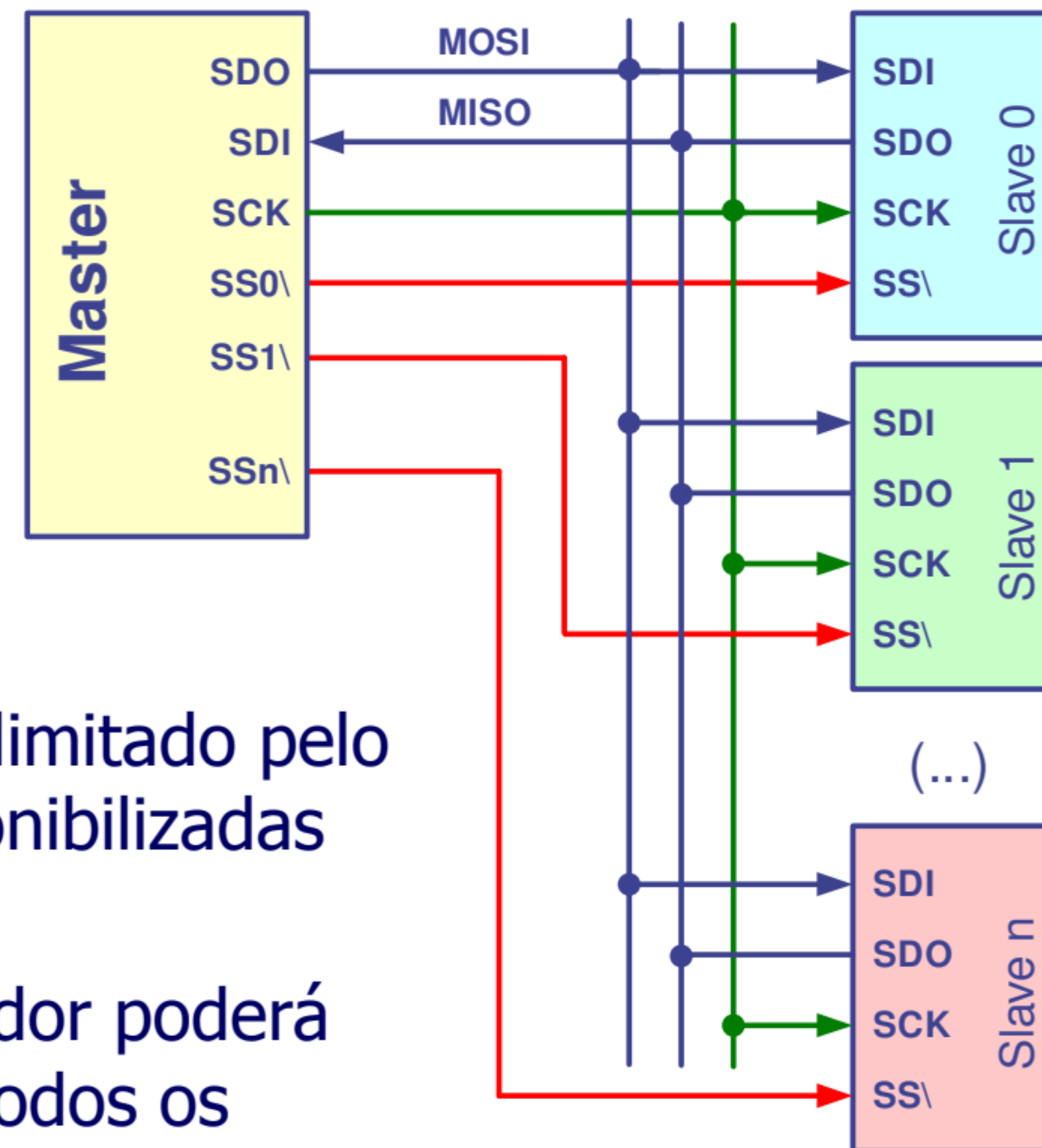
- A transição negativa do relógio é usada pelo *master* e pelo *slave* para colocar na respetiva linha de saída um bit de dados
- A transição positiva seguinte é usada pelo *master* e pelo *slave* para armazenar o bit presente na respetiva linha de entrada
- Após oito ciclos de relógio:
 - o valor inicialmente armazenado no *shift-register* do *master* foi transferido para o *shift-register* do *slave*
 - o valor inicialmente armazenado no *shift-register* do *slave* foi transferido para o *shift-register* do *master*

Operação

- O *master* ativa a linha SS\ do *slave* com o qual pretende comunicar
- O *master* ativa o relógio para sincronizar a troca de informação (em situação de repouso o relógio está inativo: fixo a 1 ou a 0)
- Em cada ciclo do relógio, por exemplo na transição positiva
 - O *master* coloca um bit na linha MOSI que será lido pelo *slave* na transição oposta seguinte
 - O *slave* coloca um bit na linha MISO que será lido pelo *master* na transição oposta seguinte
- No final da comunicação
 - O *master* desativa a linha SS\ e desativa o relógio (que fica estável, por exemplo, no nível lógico 1)
 - O relógio só permanece ativo durante a transferência de dados
- Após este processo, o *master* e o *slave* trocaram os conteúdos dos seus *shift registers*

Arquitetura de ligação

- Sinais de seleção ("slave select") independentes *→ um para cada slave*
- Em cada instante apenas um $SSx\backslash$ está ativo, isto é, apenas 1 *slave* está selecionado
- Os sinais SDO dos *slaves* (MISO) não selecionados estão em alta impedância
- O número máximo de *slaves* está limitado pelo número de linhas de seleção disponibilizadas pelo *master*
- Alternativamente, o microcontrolador poderá gerar, através de portos digitais, todos os sinais $SSx\backslash$ necessários para comunicar com os *slaves*, ultrapassando a limitação anterior



Tipos de transferências

- O SPI opera sempre em modo "full-duplex", ou seja, em cada ciclo de relógio, tanto o *master* como o *slave* enviam e recebem dados simultaneamente ("data exchange")
- No entanto, em algumas situações, os dados recebidos podem ser irrelevantes (e descartados) por um dos dispositivos
- Podem considerar-se os seguintes cenários de transferência:
- **Bidirecional** - *como está*
 - são transferidos dados válidos em ambos os sentidos (*master* → *slave* e *slave* → *master*)
- **Master → slave (operação de escrita)**
 - *master* envia dados para o *slave*
 - o *slave* também transmite dados, mas o *master* descarta-os - *no software*
- **Slave → master (operação de leitura)**
 - *master* pretende ler dados do *slave*
 - para isso transfere para o *slave* uma palavra com um valor arbitrário (por exemplo 0) *do tamanho dos dados que queremos ler (enviamos n = recebemos n bits)*
 - o *slave* recebe essa palavra, e simultaneamente transmite os dados que o *master* pretende ler (a palavra recebida pelo *slave* é descartada)

Configuração de um *master* SPI

- Antes de iniciar a transferência, o *master* deve ser configurado para ajustar os parâmetros que definem a comunicação, de acordo com as características do *slave* com o qual irá comunicar
- As principais configurações são:
 1. Configurar a frequência de relógio - porque cada slave pode ter uma freq. distinta
 2. Configurar o nível lógico de repouso ("idle") do sinal de relógio
1 ou 0 (consultar o manual)
 3. Especificar qual o flanco do relógio usado para a transmissão (a recepção é efetuada no flanco oposto) → depende do slave (consultar o seu manual)
 - Transmissão no flanco ascendente (consequentemente, a recepção é feita no flanco descendente)
 - Transmissão no flanco descendente (consequentemente, a recepção é feita no flanco ascendente)

- A interface I²C (*Inter-Integrated Circuit*)

- Características básicas e sinalização
- Endereçamento
- Transferência de dados
- Múltiplos Masters

I²C – Introdução

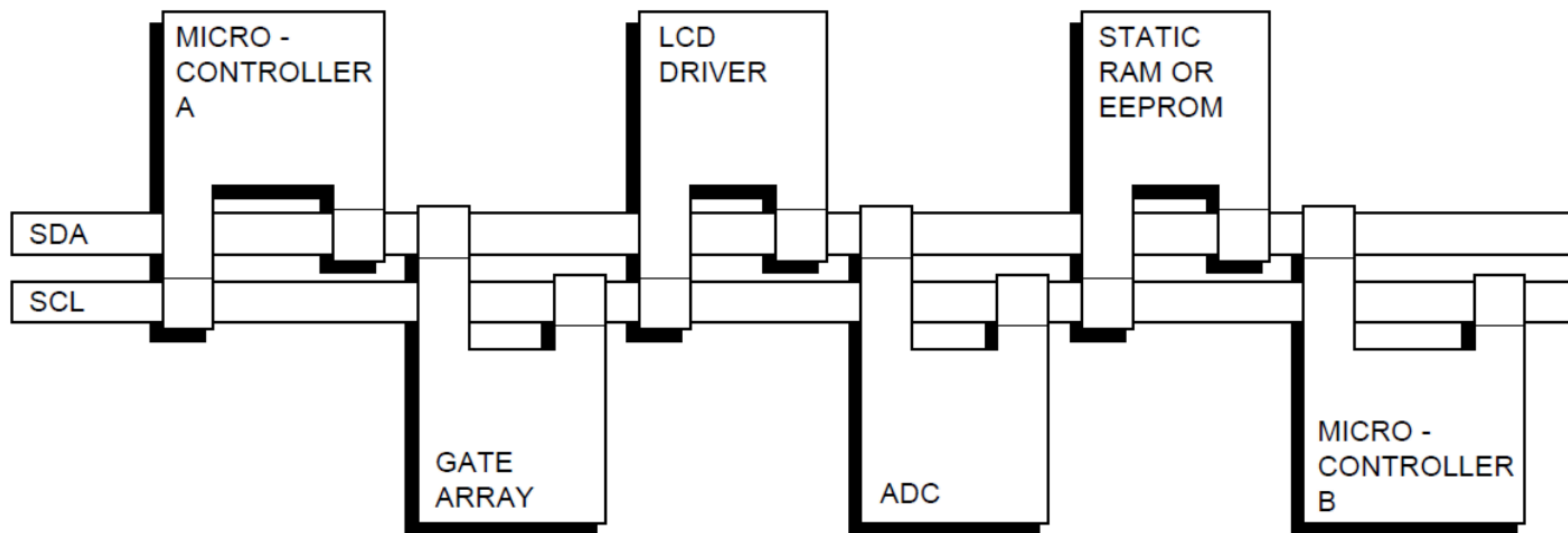
- Desenvolvido pela Philips Semiconductors (agora NXP Semiconductors)
 - Versão 1 em 1992
 - Atualmente na revisão 7 (Outubro de 2021)
- Dada a sua simplicidade, versatilidade e economia de recursos, o I²C encontra-se em diversas áreas de aplicação:
 - Sensores, DACs, ADCs
 - sensores de temperatura, sensores de pressão, acelerómetros, giroscópios, ...
 - Memória externa em microcontroladores (RAM, EEPROM/FLASH)
 - Armazenamento de dados e/ou configurações
 - Controlo de periféricos e componentes em eletrónica de consumo
 - Ecrãs OLED, LCD, ...
 - Monitorização de hardware
 - ex. temperatura de CPUs, controlo da velocidade da ventoinha em motherboards, gestão energética de baterias
 - Interface com relógios de tempo real (RTC)

I²C – Características básicas

- Comunicação série síncrona, bidirecional *half-duplex*, orientada ao byte → sempre
relógio também é transmitido
↑ canal de comunicação (usado alternadamente num sentido ou no outro)
↑ byte do transmissor para o receptor (ou vice-versa)
- A comunicação segue um modelo *master/slave*, no qual o *master* controla a transferência e um *slave* responde
 - O *master* pode atuar como transmissor ou como receptor, dependendo do contexto da comunicação
- O barramento de comunicação utiliza apenas duas linhas de sinal:
 - Serial data line (**SDA**) | Serial clock line (**SCL**)
- Cada dispositivo é endereçado por software através de um identificador único de 7 ou 10 bits, definido pelo fabricante
 - No modo "standard" (7 bits), há 128 endereços possíveis, dos quais 16 são reservados e 112 estão disponíveis para uso
- Suporta múltiplos *masters* ("multi-master"): vários masters a tentar comunicar ao mesmo tempo
 - Implementa deteção de colisões e arbitragem, garantindo que, se dois ou mais *masters* iniciarem uma transmissão simultaneamente, apenas um continuará sem que haja corrupção dos dados
o master coloca "1" na linha e se lê "0" deteta colisão e sai do processo

Exemplo de interligação num barramento I²C

S



De: I²C-bus specification and user manual, Rev. 6 — 4 April 2014

http://www.nxp.com/documents/user_manual/UM10204.pdf

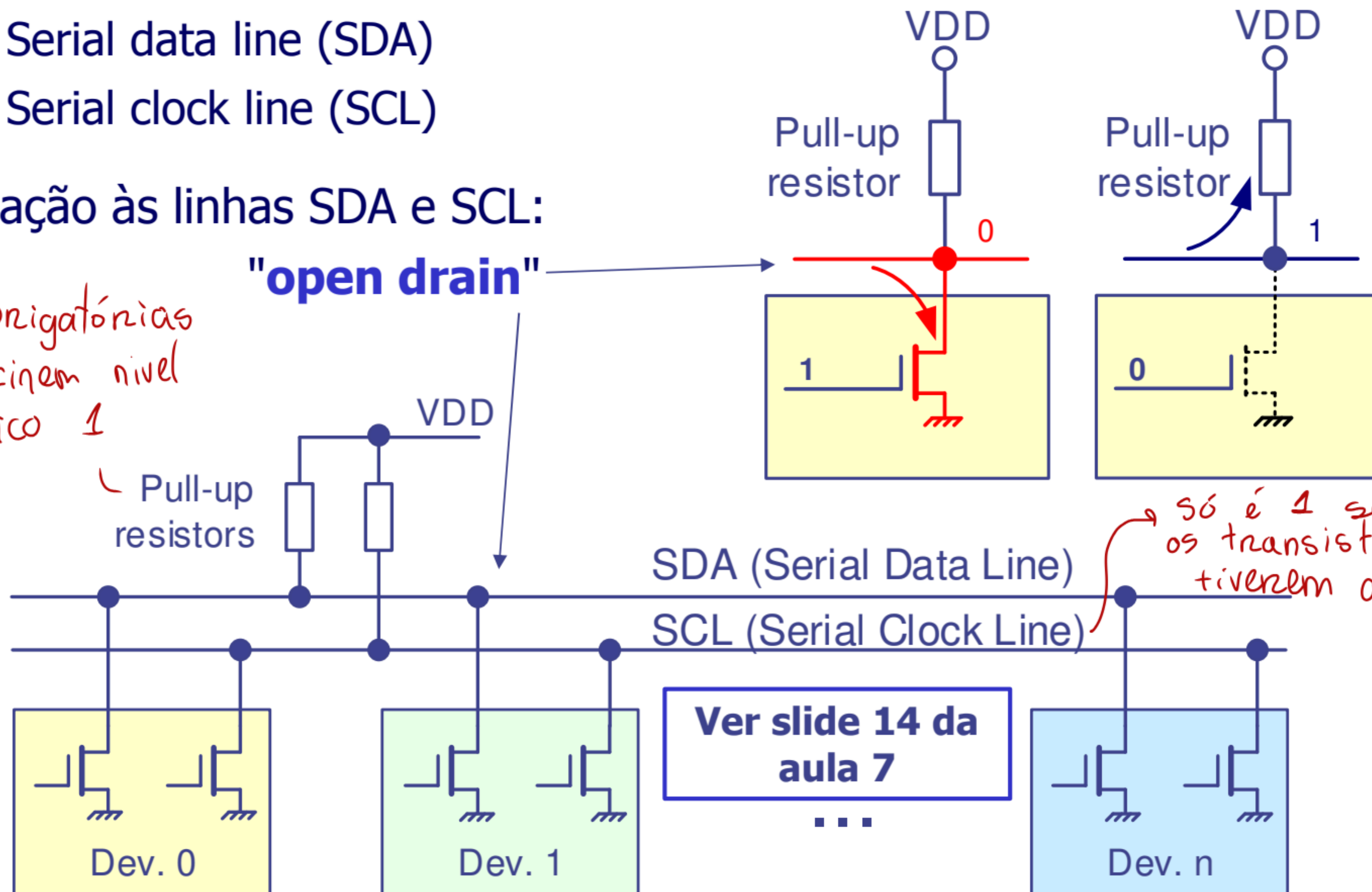
Masters e Slaves

- O *master*
 - Controla a linha SCL (Serial Clock) durante a comunicação
 - Inicia e termina a transferência de dados
 - Controla o endereçamento dos outros dispositivos – diz com quem vai comunicar
- O *slave*
 - É o dispositivo endereçado pelo *master*
 - Pode condicionar o estado da linha SCL: o *slave* pode gerar uma condição de "**clock stretch**", mantendo a linha SCL a zero para controlar o ritmo da comunicação e dar mais tempo para processar dados
forçar linha de relógio a zero, impedindo que o master envie dados (por falta de armazenamento p.ex)
- Transmissor / Recetor
 - *Master* ou *slave*
 - Um *master* transmissor envia dados para um *slave* recetor (escrita)
 - Um *master* recetor lê dados de um *slave* transmissor (leitura)

I²C – Sinalização

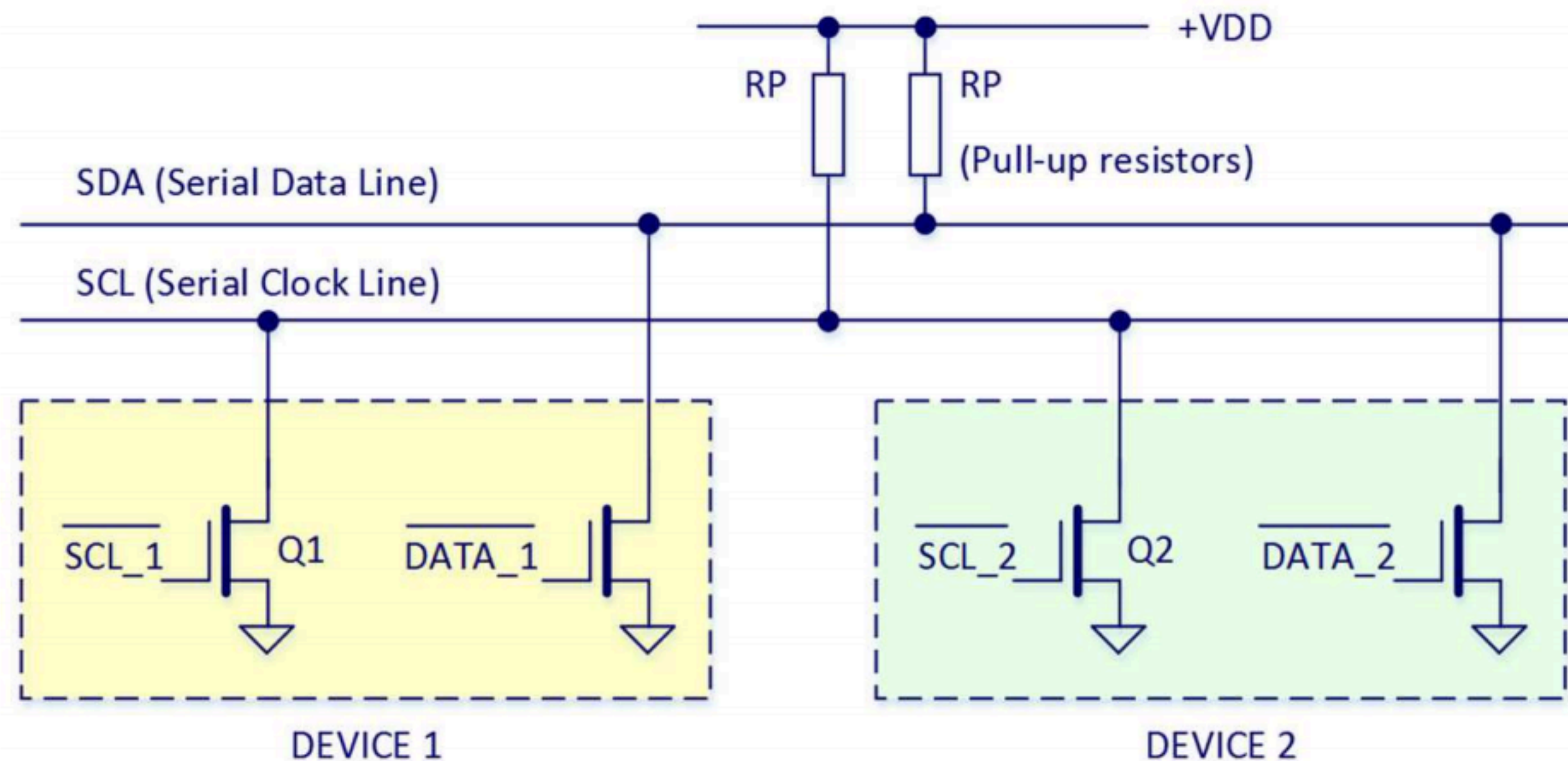
- Barramento a dois fios
 - Serial data line (SDA)
 - Serial clock line (SCL)
- Ligação às linhas SDA e SCL:

*obrigatórias
definem nível
lógico 1*



I²C – Sinalização

- As linhas de clock e dados de cada dispositivo são "wire ANDed" com os respectivos sinais do barramento



pq funciona como um AND

SCL_1	SCL_2	SCL
0 (Q1 ON)	0 (Q2 ON)	0 (D)
0 (Q1 ON)	1 (Q2 OFF)	0 (D)
1 (Q1 OFF)	0 (Q2 ON)	0 (D)
1 (Q1 OFF)	1 (Q2 OFF)	1 (R)

DATA_1	DATA_2	SDA
0 (Q1 ON)	0 (Q2 ON)	0 (D)
0 (Q1 ON)	1 (Q2 OFF)	0 (D)
1 (Q1 OFF)	0 (Q2 ON)	0 (D)
1 (Q1 OFF)	1 (Q2 OFF)	1 (R)

- Na ausência de bit dominante a linha respetiva está no nível lógico 1 (Recessivo), imposto através da resistência de *pull-up* RP
- Este esquema permite usar "bit recessivo" (1) e "bit dominante" (0) para várias sinalizações

Endereçamento

- O primeiro byte transmitido pelo *master* contém:
 - 7 bits: **endereço do *slave***
 - 1 bit: qualificação da operação (RD / WR\)
1 0
- Qualificador da operação:
 - **RD/WR\ = 0**: o *master* é o **transmissor** (escreve dados na linha SDA)
 - **RD/WR\ = 1**: o *master* é o **recetor** (lê dados da linha SDA)
- Cada *slave* lê o endereço da linha SDA; se o endereço lido coincide com o seu próprio endereço:
 - comuta para o estado "transmissor" se o bit RD/WR\ for igual a **1**
 - comuta para o estado "recetor" se o bit RD/WR\ for igual a **0**

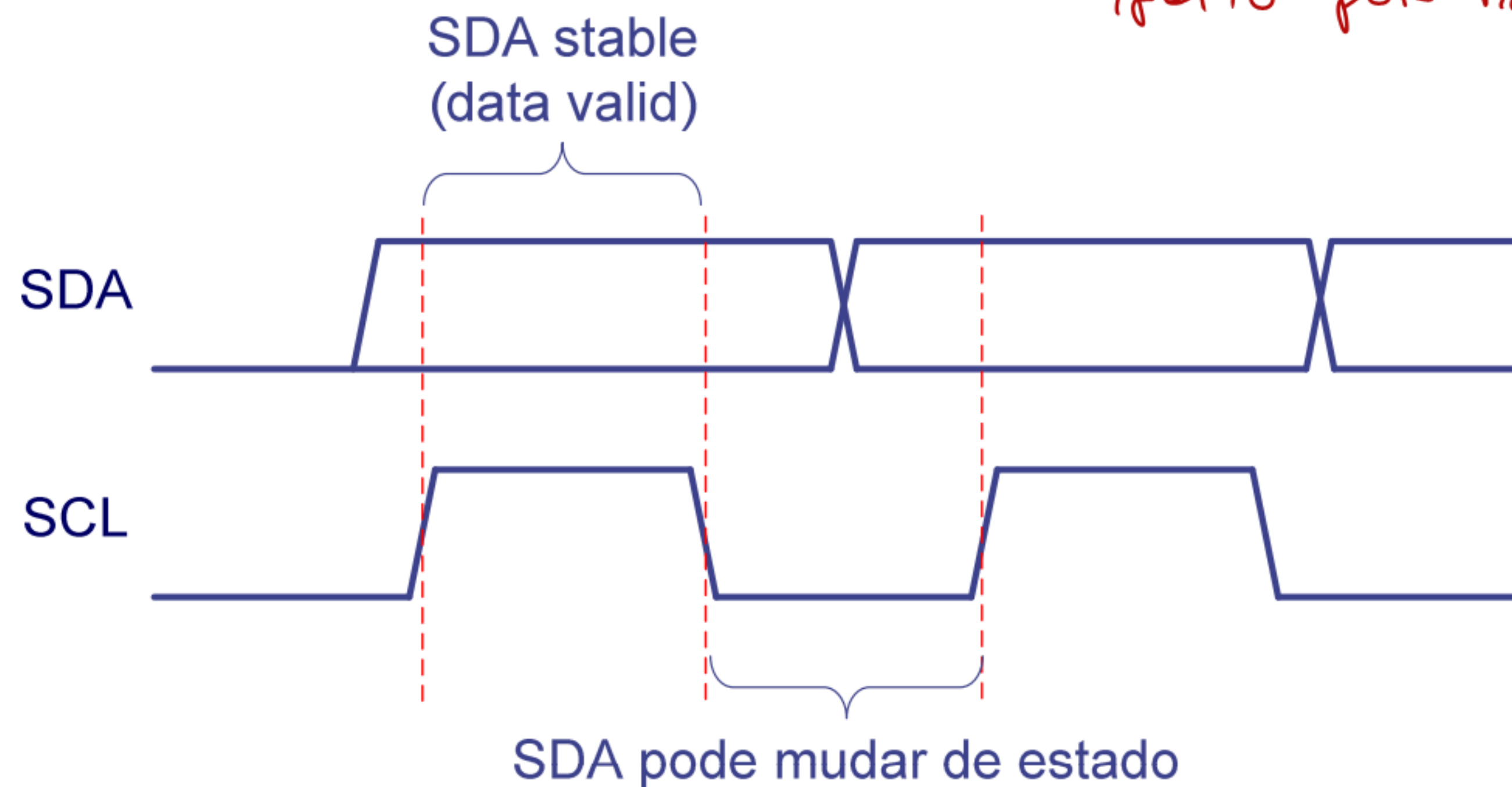
todos os slaves lêem o endereço gerado pelo master e se for o seu endereço ele passa a ser recetor ou transmissor, dependendo do sinal RD/WR

I²C – Transferência de bits

bit a bit

- Um período de relógio por bit de dados
- Dados (SDA) só são alterados quando SCL = 0
- SCL = 1: dados em SDA válidos

controlador só pode alterar a linha de dados quando o clock tá a zero (feito por hardware)

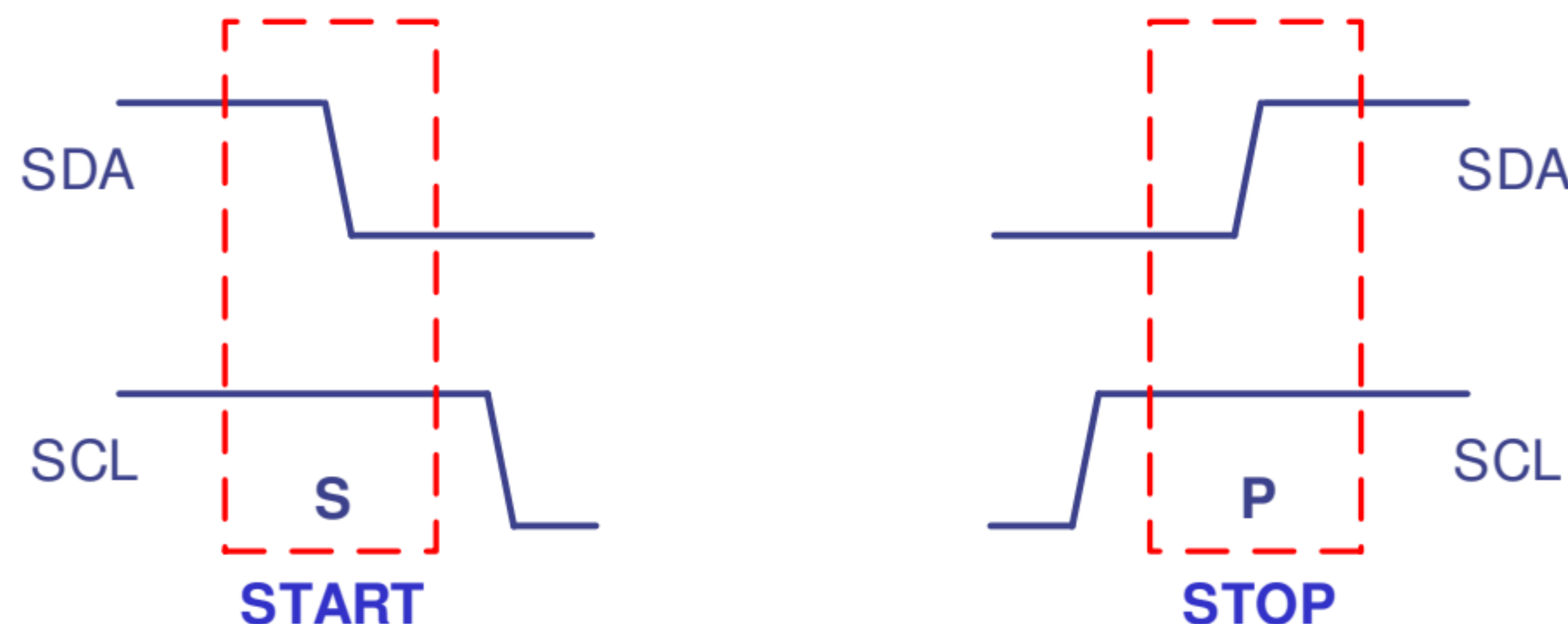


se linha de dados muda quando SCL=1

- Transições em SDA quando SCL=1 sinalizam "**condições**". As "condições" são sempre geradas pelo *master*

Símbolos (condições) - *escutados por todos os dispositivos*

- As transações são delimitadas por dois símbolos / "condições": **START** e **STOP**, geradas pelo *master*
- As condições START/STOP são sinalizadas por meio de uma sequência que viola as regras normais de transferência de dados



- Condição **START**: Transição de 1→0 em SDA quando SCL = 1
- Condição **STOP**: Transição de 0→1 em SDA quando SCL = 1
- Estado do barramento:
 - **Ocupado**: após um START (S) ou START repetidos (Sr), até ao próximo STOP
 - **Livre**: após um STOP (P), até ao próximo START

*para saber
se tá ocupado
ou livre*

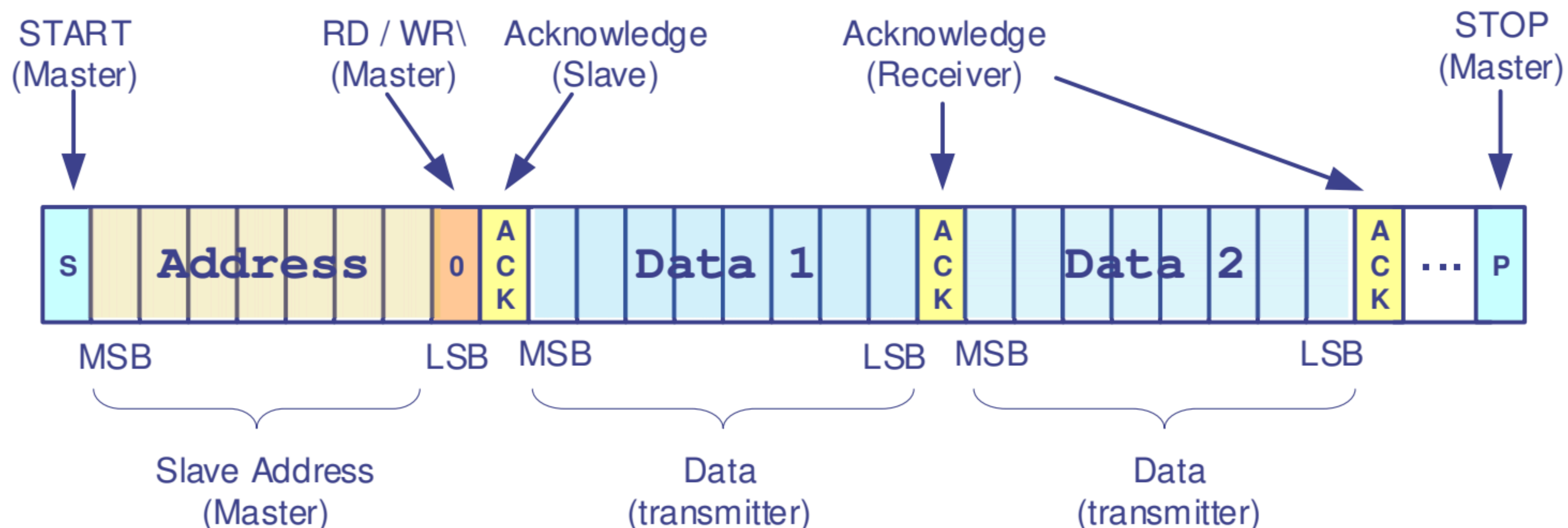
Livre = pode mudar de master

Transferência de dados

- A transferência é orientada ao byte (8 bits) sendo transmitido, em primeiro lugar, o bit mais significativo (MSbit)
- No início de uma transferência, o *master*:
 - Envia um **START** (S)
 - De seguida envia o **endereço** do slave (7 bits) e o bit de qualificação da operação (Read / Write\ - RD/WR\). *e dps coloca a linha a 1*
- Após o 8º bit (o LSbit, correspondente ao bit RD/WR\), o *slave* endereçado gera um *acknowledge* (**ACK**) na linha SDA, **sob a forma de um bit dominante** (0). *ativo a 0* *para dizer que reconheceu o endereço*
- A seguir o transmissor (*master* ou *slave*) envia 1 byte de dados
- Após o 8º bit (o LSbit), o recetor gera um acknowledge (**ACK**) na linha SDA, **sob a forma de um bit dominante** (0)
- Este ciclo de 9 bits repete-se para cada byte de dados que é transferido. *START, endereço e RD/WR + ACK, byte de dados + ACK, byte + ACK, STOP*
- No final da transferência o *master* envia um **STOP** (P). *9*

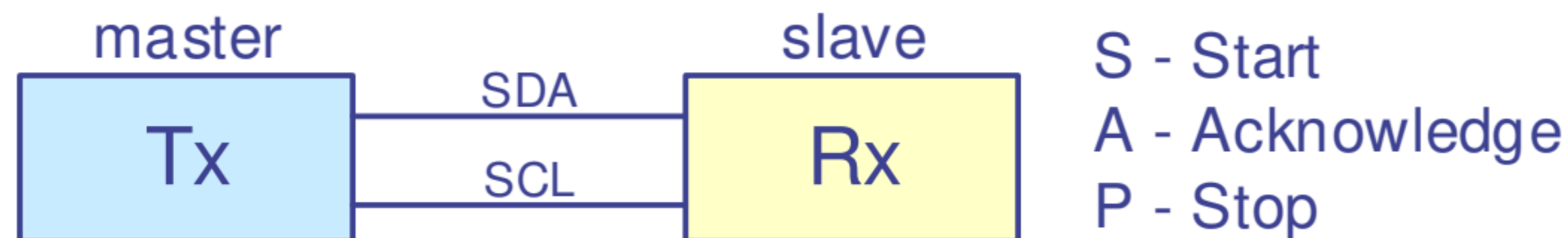
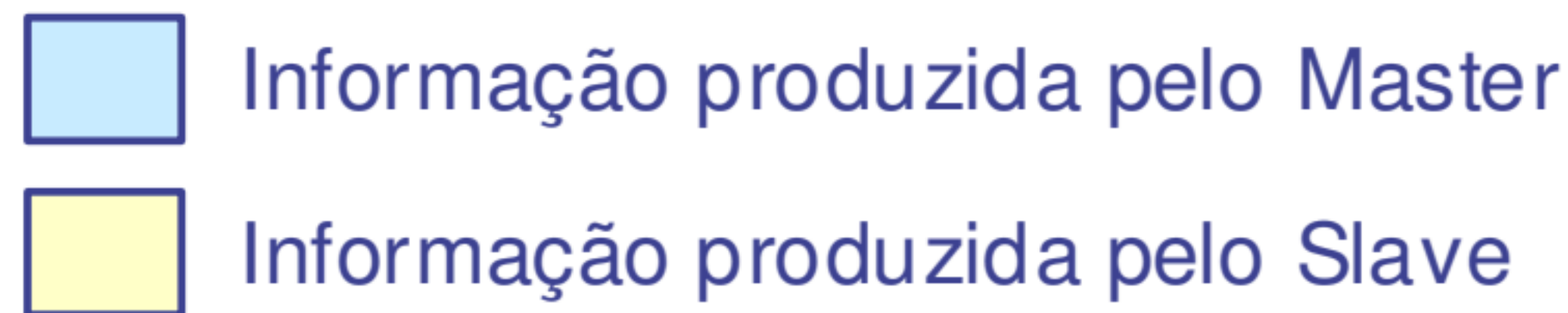
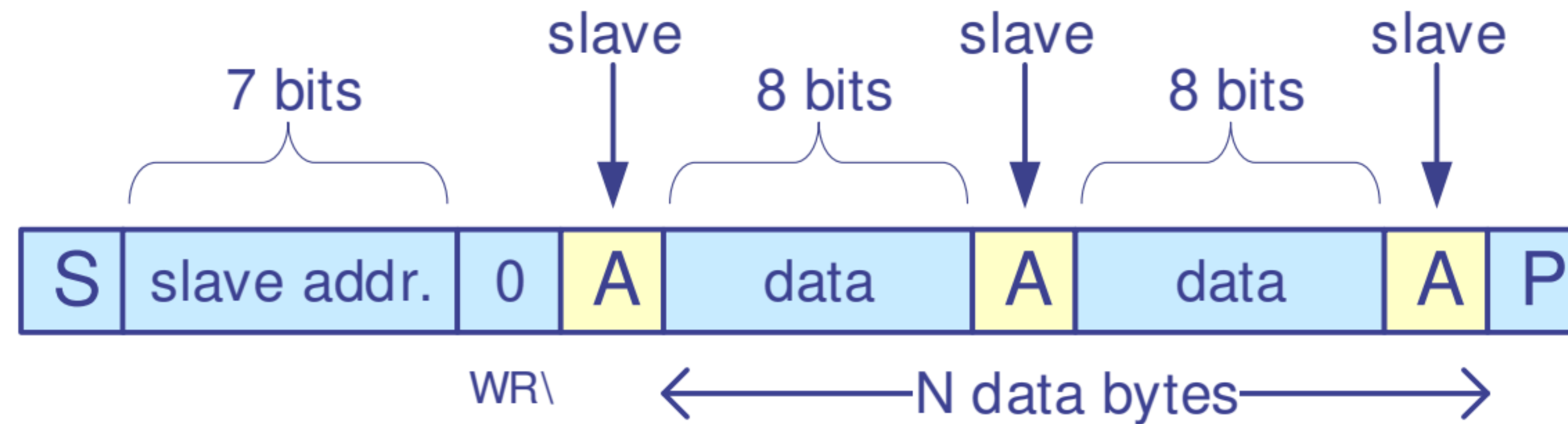
Transferência de dados – escrita

- O *master*:
 - Envia um **START** (S)
 - De seguida envia o **endereço** do slave (7 bits) e o bit de qualificação da operação (Read / Write\ - RD/WR\ = 0) *→ escrita*
- O *slave* endereçado faz o **acknowledge** (ACK) na *slot* seguinte
- Nos 8 ciclos de relógio seguintes o *master* envia o byte de dados e no 9º ciclo de relógio avalia o **acknowledge do slave**. Este ciclo de 9 bits repete-se para cada byte de dados que o *master* pretenda transferir.



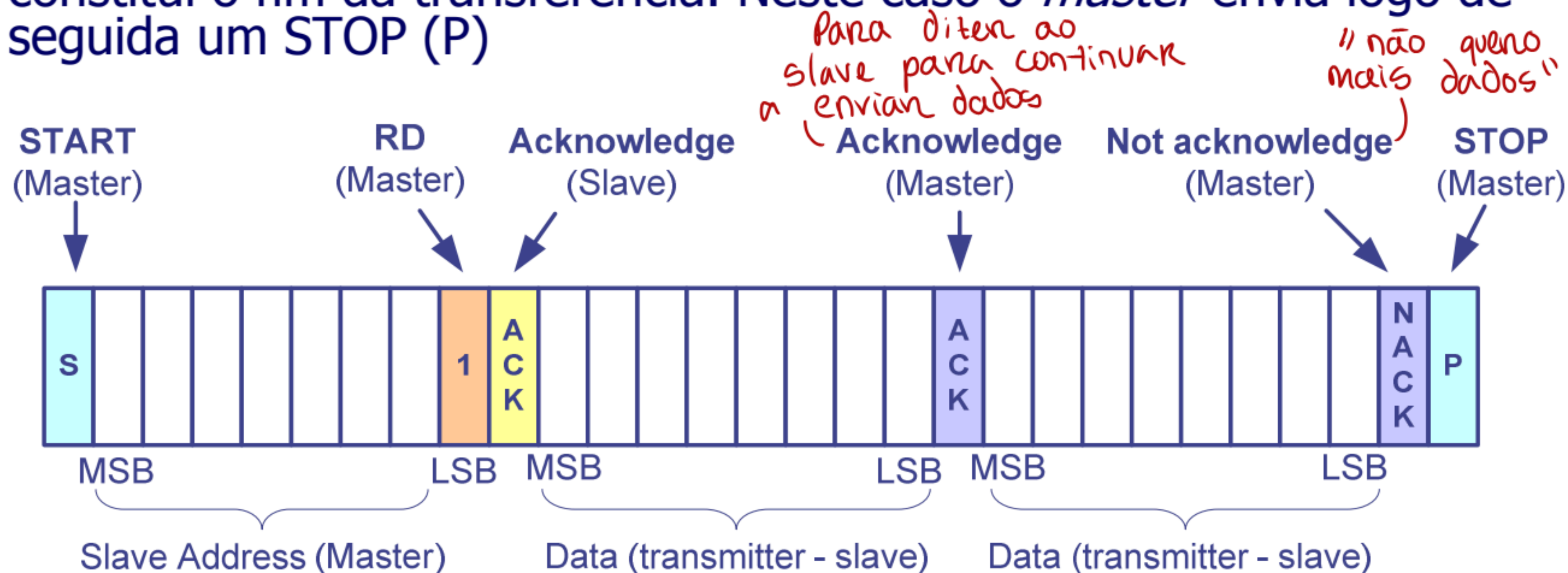
Transferência de dados – escrita

- Operação de **Escrita** (*master* é transmissor, *slave* é recetor)



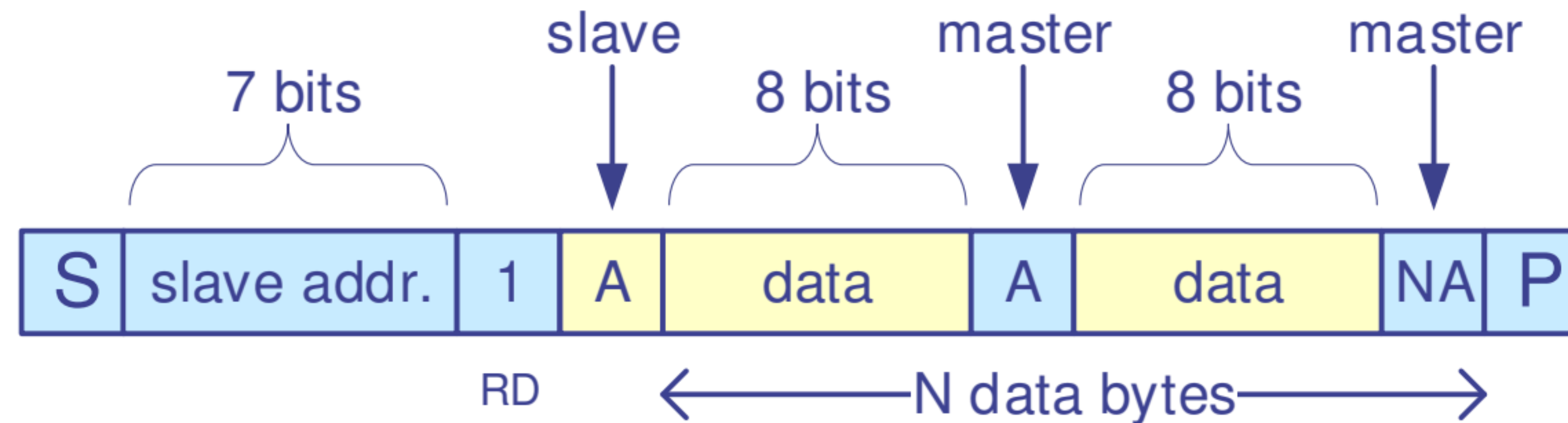
Transferência de dados – leitura

- O *master*:
 - Envia um **START** (S)
 - De seguida envia o **endereço** do slave (7 bits) e o bit de qualificação da operação (Read / Write\ - RD/WR\ = 1) *leitura*
- O *slave* endereçado faz o **acknowledge** (ACK) na *slot* seguinte
- O *slave* envia 8 bits de dados
- No final da sequência de 8 bits, o *master* (que é recetor) gera:
 - um **acknowledge**, sinalizando que vai continuar a ler dados, ou
 - um **not acknowledge** informando o *slave* de que o byte recebido constitui o fim da transferência. Neste caso o *master* envia logo de seguida um STOP (P)



Transferência de dados – leitura

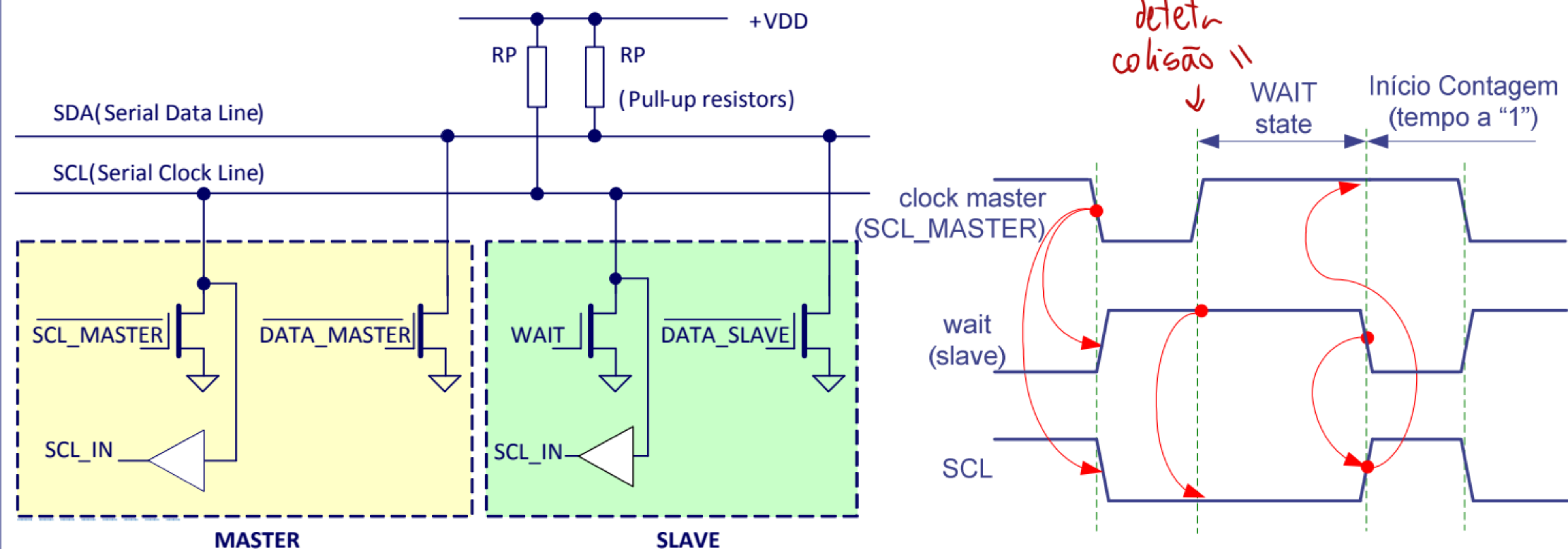
- Operação de **Leitura** (*master* é recetor, *slave* é transmissor)



- O "not acknowledge" (NA) enviado pelo *master* sinaliza o *slave* do fim da transferência

Transferência de dados – "clock stretching"

- O *slave* pode forçar o alargamento da transferência mantendo a zero (bit dominante) a linha SCL; esta técnica designa-se por "clock stretching"
- O sinal "wait" (do *slave*) condiciona o estado da linha SCL do barramento: enquanto "wait" estiver a 1, a linha SCL está forçada a nível lógico 0 (dominante)

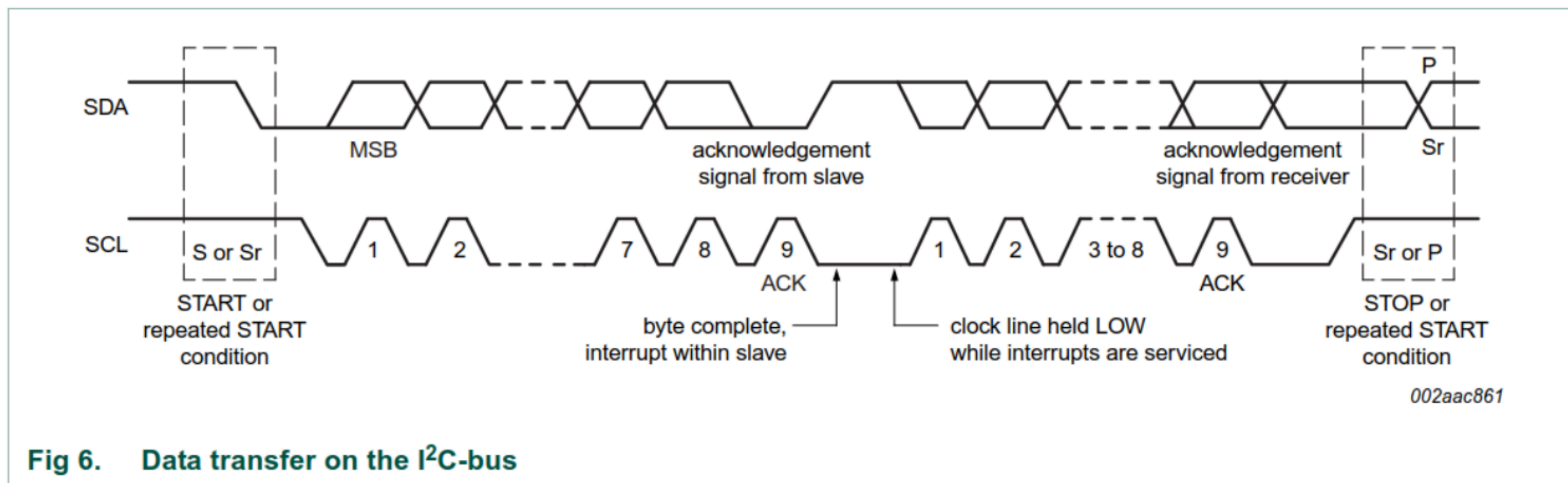


- *master* fica em "wait state" enquanto **SCL_MASTER** $\neq$ **SCL_IN**

Transferência de dados - exemplo

S

- Exemplo de uma sequência de transferência:



De: I²C-bus specification and user manual, Rev. 6 — 4 April 2014

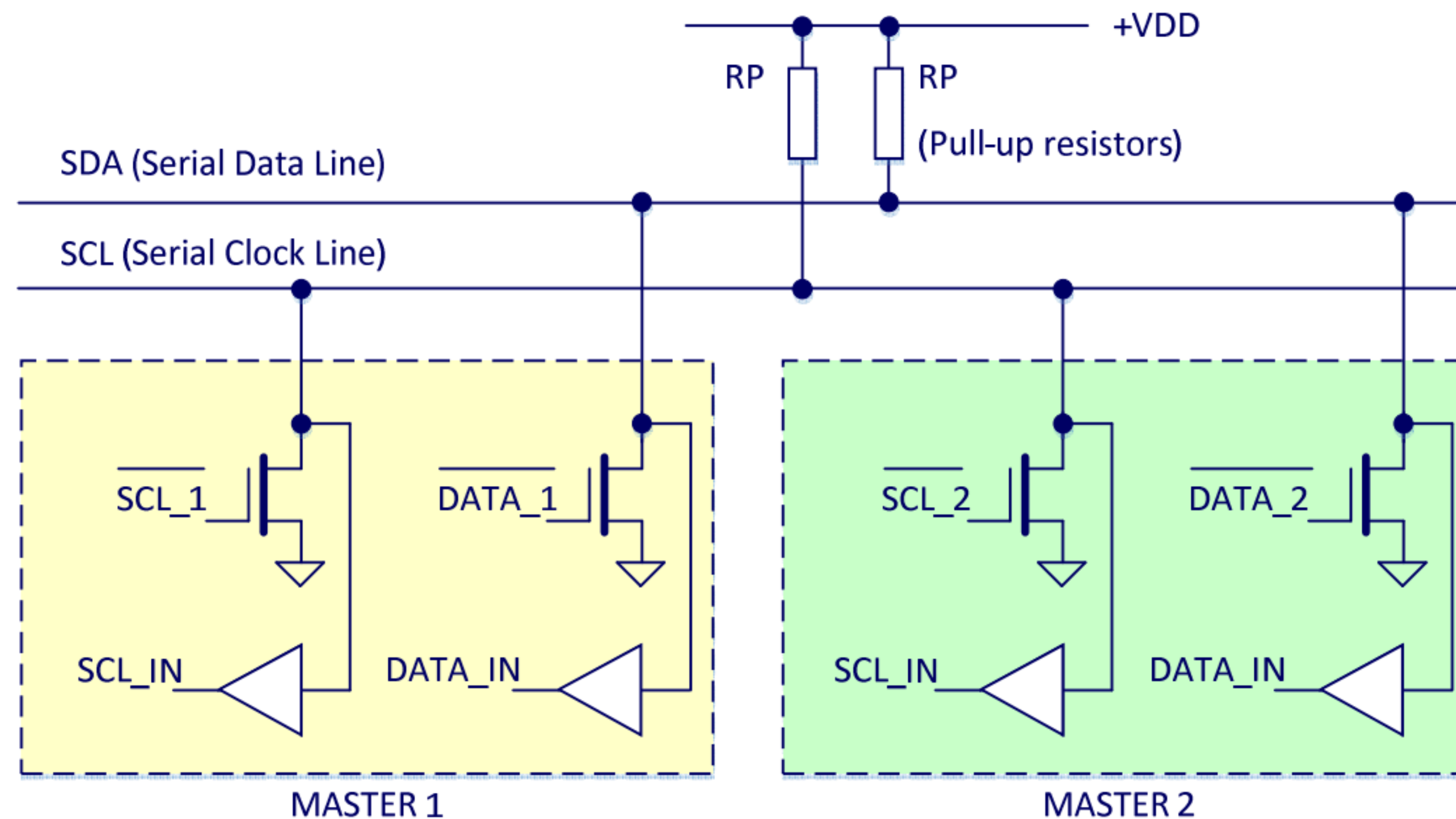
Múltiplos *masters*

- Dois (ou mais) *masters* podem iniciar uma transmissão simultânea num barramento livre (isto é, após um STOP)
- É necessário um método para decidir qual dos *masters* toma o controlo do barramento e completa a transmissão – este processo é conhecido como **arbitragem de acesso ao barramento**
- O *master* que perde o processo de arbitragem retira-se e só tenta novo acesso ao barramento na próxima situação de "barramento livre" (após um STOP)
- Na gestão do acesso ao barramento é necessário:
 1. garantir que os relógios dos *masters* estão **sincronizados**
 2. um processo que defina qual o *master* que ganha o acesso ao barramento, i.e., que controla a linha SDA (**arbitragem**)
↳ Através do START e STOP
- A sincronização de relógios e a arbitragem são baseados na técnica bit dominante/bit recessivo:
 - Nível lógico "0" – bit dominante
 - Nível lógico "1" – bit recessivo (é anulado por um bit dominante)

Arbitragem (linha SDA)

- Quando o barramento está livre ("idle") dois ou mais *masters* podem iniciar uma transferência; todos geram START resultando numa condição de START válida no barramento *fica o master que "ganha a corrida"*
o master que envia e lê o mesmo
 - Recordando: **arbitragem** é o mecanismo que assegura que, se mais do que um *master* tentar controlar o barramento simultaneamente, apenas um poderá continuar, sem que haja perturbação da comunicação iniciada pelo *master* vencedor, *os outros podem passar a funcionar como slaves*
- A arbitragem é feita por bit dominante / bit recessivo e **processa-se bit a bit**
- Por cada novo bit enviado, quando a linha SCL está a 1 cada *master* lê a linha SDA e verifica se o seu valor coincide com o que enviou:
 - O processo de arbitragem é perdido por um *master* quando lê da linha nível lógico 0 (dominante) tendo enviado nível lógico 1 (recessivo)
- O *master* que perde o processo de arbitragem
 - Retira-se, libertando a linha SDA (comuta de imediato para modo *slave*)
 - Tenta de novo quando o barramento passar ao estado "idle" (espera o aparecimento de uma condição STOP)

Arbitragem (linha SDA)



DATA_1	DATA_2	SDA
0	0	0 (D)
0	1	0 (D)
1	0	0 (D)
1	1	1 (R)

Arbitragem continua

Master 2 perde arbitragem

Master 1 perde arbitragem

Arbitragem continua

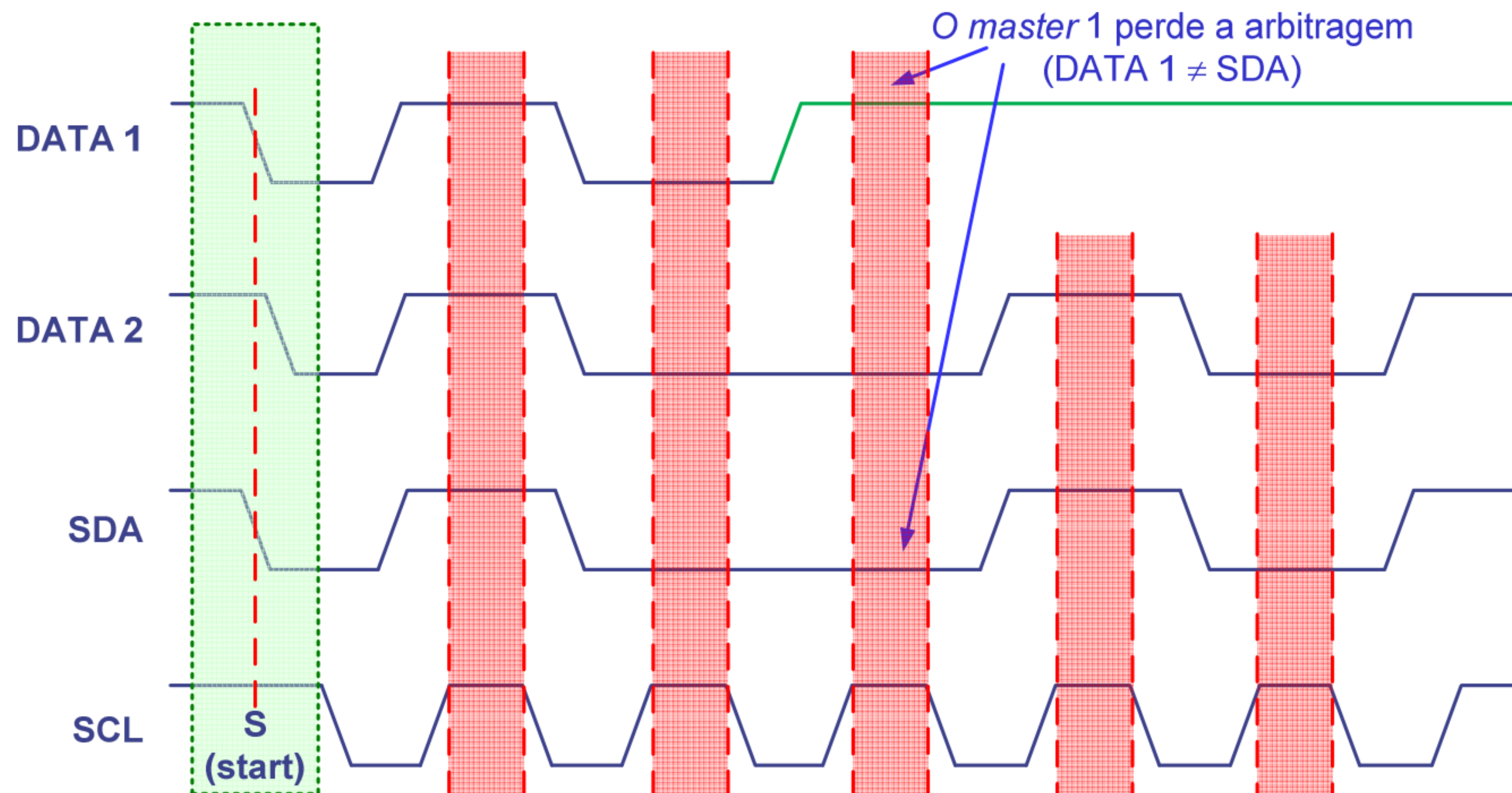
master 1 escreveu 0 e levou 0

se quem os dois escrevem no mesmo slave ganha quem escrevem nele os dados de valor mais baixo (se ambos quiserem escrever o mesmo ficam todos felizes)

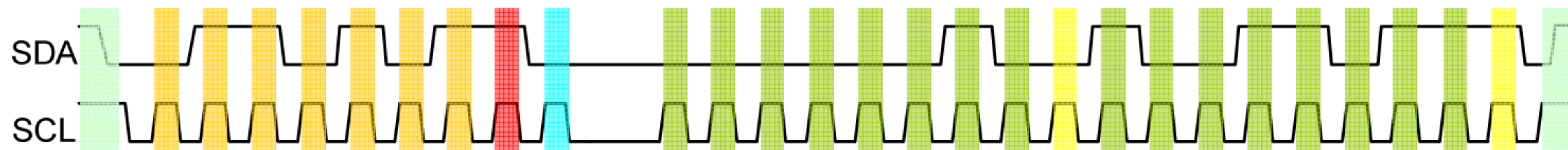
Arbitragem (linha SDA) – Exemplo

- Exemplo:

- 1) o *master 1* e o *master 2* iniciam uma transmissão (quando o barramento passa a "idle")
- 2) o *master 1* perde a arbitragem na transmissão do 3º bit



Exercício



- Considere o diagrama temporal acima representado. Admita que representa a comunicação entre um *master* (μ C) e um *slave* (ADC de 10 bits).
1. Qual o endereço do elemento *slave* (ADC)?
 2. Estamos perante uma operação de escrita ou de leitura?
 3. Quantos ACKs são gerados pelo *slave*?
 4. Quantos ACKs são gerados pelo *master*?
 5. Quantos NACKs são gerados? Por quem?
 6. Qual o valor (expresso em hexadecimal) que foi fornecido pela ADC ao μ C, sabendo que este começa sempre pelo MSBit?
 7. Quantas situações de *clock stretch* são gerados nesta transação? Por quem?
 8. Supondo que a frequência do relógio é de 1MHz e que o *stretch* corresponde a dois ciclos de relógio, qual a duração total da transação?

Anexo

• Endereços reservados.

Two groups of eight addresses (0000 XXX and 1111 XXX) are reserved for the purposes shown in [Table 3](#).

Table 3. Reserved addresses

X = don't care; 1 = HIGH; 0 = LOW.

Slave address	R/W bit	Description
0000 000	0	general call address ^[1]
0000 000	1	START byte ^[2]
0000 001	X	CBUS address ^[3]
0000 010	X	reserved for different bus format ^[4]
0000 011	X	reserved for future purposes
0000 1XX	X	Hs-mode master code
1111 1XX	1	device ID
1111 0XX	X	10-bit slave addressing

[1] The general call address is used for several functions including software reset.

[2] No device is allowed to acknowledge at the reception of the START byte.

[3] The CBUS address has been reserved to enable the inter-mixing of CBUS compatible and I²C-bus compatible devices in the same system. I²C-bus compatible devices are not allowed to respond on reception of this address.

[4] The address reserved for a different bus format is included to enable I²C and other protocols to be mixed. Only I²C-bus compatible devices that can work with such formats and protocols are allowed to respond to this address.

De: I²C-bus specification and user manual, Rev. 6 — 4 April 2014